

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: A. Suraj varma

Reg.No: 192525339

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5

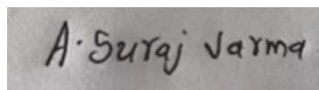
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I A. Suraj varma (192525339) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Industry Problem Solving Task

1. E-Commerce Product Records – 10 Million Records

Question: An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Answer

For an e-commerce company storing **10 million product records**, I recommend a **Sorted File Organization combined with B+ Tree indexing**.

Recommended Strategy

The product records can be stored in sorted order based on Product_ID.

A B+ Tree index can then be created on Product_ID for fast product retrieval.

Example:

Product_ID | Product_Name | Category | Price

```

-----
10001      | Laptop      | Computer | 55000
10002      | Mouse       | Computer | 800
10003      | Keyboard    | Computer | 1500

```

Why this strategy?

E-commerce systems commonly require:

- Fast search by Product ID.
- Range queries such as products priced between ₹10,000 and ₹50,000.
- Sequential retrieval of products.
- Efficient searching over millions of records.

A B+ Tree provides approximately **$O(\log n)$** search, insertion, and deletion.

Cost Analysis

Operation	Heap File	Sorted + B+ Tree
Search	$O(n)$	$O(\log n)$
Insert	$O(1)$	$O(\log n)$
Delete	$O(n)$	$O(\log n)$
Range Search	Slow	Very efficient
Storage	Low	Higher due to index

Although maintaining the sorted file and B+ Tree requires additional storage and update cost, the benefit is much faster retrieval for **10 million records**.

Conclusion

Sorted File + B+ Tree Index is the most suitable choice because the company requires efficient product search and range retrieval. The additional index-storage cost is justified by the significant reduction in search time.

2. Banking System – Account Number and Date Range

Question: A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Answer

For a banking transaction table, I recommend a **composite B+ Tree index** on:

(Account_Number, Transaction_Date)

Example table:

Transaction

Transaction_ID

Account_Number

Transaction_Date

Transaction_Type

Amount

Index

CREATE INDEX idx_account_date

ON Transaction(Account_Number, Transaction_Date);

How it works

Suppose the query is:

SELECT *

FROM Transaction

WHERE Account_Number = 10025

AND Transaction_Date BETWEEN '2026-01-01' AND '2026-01-31';

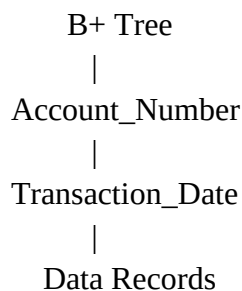
The database first locates the account number and then efficiently searches the required date range.

Why B+ Tree?

B+ Trees are suitable because:

- Account numbers can be searched efficiently.
- Dates are ordered inside the index.
- Range queries are efficient.
- Leaf nodes are linked, which helps sequential range retrieval.

Recommended Index Design



Conclusion

A **composite B+ Tree index on (Account_Number, Transaction_Date)** is the best choice because it supports both exact account-number searches and date-range searches efficiently.

3. Healthcare System – PatientID and Doctor Name

Question: A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Answer

The healthcare system has **two different search requirements**:

1. Retrieve a patient using PatientID.
2. Retrieve patients using Doctor_Name.

Therefore, separate indexes should be created.

Primary Index

Create a **B+ Tree primary index** on PatientID.

CREATE UNIQUE INDEX idx_patient_id

ON Patient(PatientID);

Because PatientID is unique, this provides fast direct retrieval.

Secondary Index

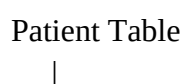
Create a **secondary B+ Tree index** on Doctor_Name.

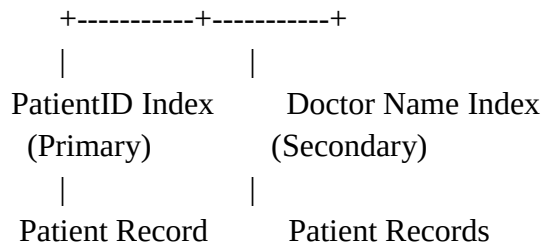
CREATE INDEX idx_doctor_name

ON Patient(Doctor_Name);

This allows the database to find all patients associated with a particular doctor.

Multi-Level Structure





For very large datasets, each B+ Tree can itself have multiple levels:

Level 1: Root

Level 2: Intermediate Nodes

Level 3: Leaf Nodes

Records

Conclusion

Use a **primary B+ Tree index on PatientID** and a **secondary B+ Tree index on Doctor_Name**. This provides efficient access through both search attributes while keeping the indexing structure scalable for a large healthcare database.

4. Logistics – B-Tree vs B+ Tree

Question: A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Answer

For a logistics company with frequent shipment insertions, deletions, and searches, a **B+ Tree is more suitable than a B-Tree**.

Comparison

Feature	B-Tree	B+ Tree
Data stored	Internal + leaf nodes	Leaf nodes
Search	Efficient	Efficient
Insert	Efficient	Efficient
Delete	Efficient	Efficient
Range query	Less efficient	Highly efficient
Leaf nodes linked	No	Yes
Database systems	Less common	Commonly preferred

Why B+ Tree?

A logistics system may need queries such as:

Find shipment between ID 1000 and 2000

B+ Tree leaf nodes are linked:

[1000,1010,1020]

↓

[1030,1040,1050]

↓

[1060,1070,1080]

Once the first matching leaf is found, the database can traverse the linked leaves.

Dynamic Operations

For insertion:

Insert → Find correct leaf → Insert → Split if required

For deletion:

Delete → Remove from leaf → Rebalance if required

Conclusion

B+ Tree is the better choice because it handles dynamic insertion and deletion efficiently while providing excellent support for range queries, which are common in logistics systems.

5. HR System – 5,000 Employees

Question: Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing.

Answer

For an HR system containing **5,000 employees**, hashing can provide fast retrieval using Employee_ID.

A suitable hash function is:

$h(\text{Employee_ID}) = \text{Employee_ID} \bmod M$

where M is the number of buckets.

For example:

$M = 1000$

Then:

$h(50235) = 50235 \bmod 1000$
 $= 235$

Static Hashing

In static hashing, the number of buckets is fixed.

Advantages:

- Simple implementation.
- Fast average search.
- Low management overhead.

Disadvantage:

- If the number of employees grows significantly, overflow buckets may increase.

Extendible Hashing

Extendible hashing dynamically expands the directory when buckets overflow.

Advantages:

- Handles growth dynamically.
- Reduces long overflow chains.
- Efficient for insertions and deletions.
- Suitable when the employee database changes frequently.

Comparison

Feature	Static Hashing	Extendible Hashing
Bucket size	Fixed	Dynamic
Growth handling	Poorer	Excellent
Overflow	Overflow chains	Bucket splitting
Directory	Fixed	Can double
Complexity	Simple	More complex
Best for	Stable data	Growing data

Recommendation

For an HR system, I recommend **Extendible Hashing** if employee records are expected to grow or change frequently.

If the organization knows that approximately 5,000 records will remain stable, **static hashing** is simpler and sufficient.

Conclusion

Because HR databases can grow over time, **extendible hashing is the preferred practical solution**, while static hashing is suitable for a fixed-size employee database.

6. Airline Reservation System

Question: An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Answer

The airline system has two different requirements:

1. **Point query:** Search using Flight_Number.
2. **Range query:** Search flights within a date range.

Therefore, I recommend **two B+ Tree indexes**.

Index 1 – Flight Number

```
CREATE UNIQUE INDEX idx_flight_number  
ON Flight(Flight_Number);
```

This supports fast exact searches:

```
SELECT *  
FROM Flight  
WHERE Flight_Number = 'AI202';
```

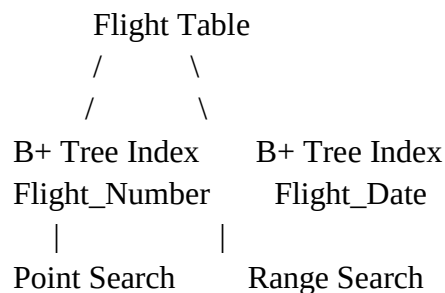
Index 2 – Flight Date

```
CREATE INDEX idx_flight_date  
ON Flight(Flight_Date);
```

This supports:


```
SELECT *
FROM Flight
WHERE Flight_Date BETWEEN '2026-08-01' AND '2026-08-07';
```

Combined Structure



Why B+ Tree?

B+ Tree supports:

- Exact-match searching.
- Ordered data.
- Range queries.
- Efficient sequential traversal through linked leaves.

Conclusion

Use a **B+ Tree index on Flight_Number** for point queries and another **B+ Tree index on Flight_Date** for range queries. This provides an efficient and practical indexing solution for airline reservations.

7. University Student Database

Question: For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Answer

For a university student database, I recommend:

- **Sorted file organization based on Student_ID**
- **Secondary B+ Tree index on Department**
- **Secondary B+ Tree index on CGPA**

Example:

Student

Student_ID

Name

Department

CGPA

File Organization

The main student records can be stored in sorted order by:

Student_ID

This provides organized storage and efficient primary-key access.

Secondary Index 1 – Department

```
CREATE INDEX idx_department
```

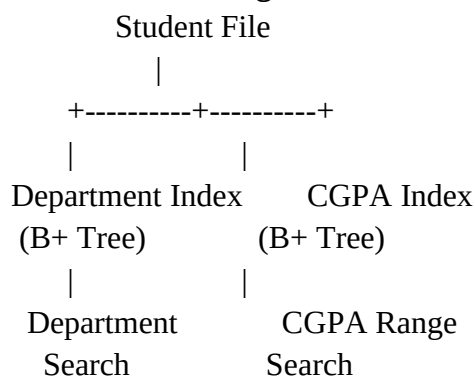
ON Student(Department);
 This supports queries such as:
 SELECT *
 FROM Student
 WHERE Department = 'CSE';

Secondary Index 2 – CGPA

CREATE INDEX idx_cgpa
 ON Student(CGPA);

This supports:
 SELECT *
 FROM Student
 WHERE CGPA BETWEEN 8.0 AND 9.0;

Recommended Design



Conclusion

A sorted file based on Student_ID with secondary B+ Tree indexes on Department and CGPA provides efficient exact and range queries while keeping the primary data organized.

8. Disk Block Utilization

Question: Analyze disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Given

Number of records = 100,000

Record size = 50 bytes

Block size = 4 KB

Assume:

1 KB = 1024 bytes

Therefore:

Block size = 4×1024
 = 4096 bytes

Step 1 – Records per Block

Records per block = $4096 / 50$
 = 81.92

Therefore, only **81 complete records** fit in one block.

Step 2 – Total Blocks

Total blocks = $\text{ceil}(100000 / 81)$
= 1235 blocks

Step 3 – Actual Data Stored

100000×50

= 5,000,000 bytes

The final block may not be completely filled.

Comparison

File Organization	Number of Blocks	Space Utilization	Main Characteristic
Heap	1235	High	Records stored as inserted
Sorted	1235	High	Records maintained in sorted order
Hashed	1235+*	Depends on overflow	Records distributed using hash

*In a hashed organization, additional overflow blocks may be required depending on the hash function, bucket size, and load factor.

Important Analysis

The basic record storage requirement is approximately:

1235 blocks

for all three organizations if the same blocking factor is used.

However, their **effective utilization differs**:

- **Heap:** Can have good space utilization but may suffer from fragmentation depending on deletion strategy.
- **Sorted:** Data blocks can remain densely packed, but insertions may require movement/reorganization.
- **Hashed:** Buckets may contain unused space or overflow blocks because records are distributed according to the hash function.

Conclusion

The raw storage requirement is approximately **1235 blocks**, but the practical utilization depends on insertion, deletion, bucket allocation, and overflow handling. For static data, sorted organization can provide good utilization; for fast equality searches, hashing is more suitable.

9. Social Media – 500 Million Posts

Question: A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Answer

For **500 million posts**, a single index is not sufficient because the system has three major query attributes:

user_id

timestamp

hashtag

I recommend a combination of **composite B+ Tree indexes**.

Index 1 – User and Timestamp

```
CREATE INDEX idx_user_time  
ON Posts(user_id, timestamp);
```

This is useful for queries such as:

```
SELECT *  
FROM Posts  
WHERE user_id = 1001  
ORDER BY timestamp;
```

It also supports:

```
WHERE user_id = 1001  
AND timestamp BETWEEN '2026-01-01' AND '2026-01-31';
```

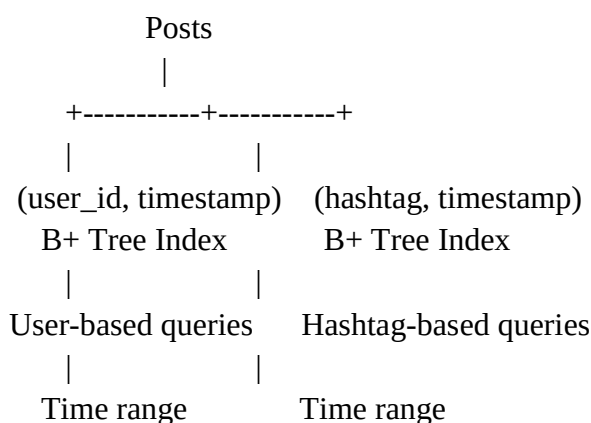
Index 2 – Hashtag and Timestamp

```
CREATE INDEX idx_hashtag_time  
ON Posts(hashtag, timestamp);
```

This supports:

```
SELECT *  
FROM Posts  
WHERE hashtag = '#technology'  
AND timestamp BETWEEN '2026-01-01' AND '2026-01-31';
```

Recommended Structure



Why composite indexes?

The first column has the highest filtering importance for the corresponding query.

For example:

(user_id, timestamp)

is more useful for:

user_id + time range

than creating unrelated single-column indexes.

Scalability

With **500 million records**, index size can become very large. Therefore, practical deployment should also consider:

- Partitioning by time.
- Index maintenance cost.
- Storage overhead.

- High write volume.
- Archiving older posts.

Conclusion

A **composite B+ Tree index on (user_id, timestamp)** and another on **(hashtag, timestamp)**, combined with time-based partitioning for scalability, provides an efficient solution for the three required access patterns.

10. Supermarket Billing System – 1 Million Daily Transactions

Question: Compare the performance of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Answer

A supermarket billing system performs many transaction operations every day. Therefore, the file organization must support:

- Fast insertion of new bills.
- Fast retrieval of transactions.
- Deletion or correction of records when required.
- Large transaction volume.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	O(1) average	O(n) / costly	O(1) average
Search by key	O(n)	O(log n) with binary search/index	O(1) average
Delete	O(n)	O(n) / restructuring	O(1) average
Range search	Poor	Excellent	Poor
Maintaining order	No	Yes	No
Overflow	No normal overflow	No	Possible
Best use	Heavy insert workload	Ordered/range queries	Exact-key queries

Heap File

New transactions can simply be appended.

New Bill → End of File

Advantage: Very fast insertion.

Disadvantage: Searching for a particular transaction may require scanning the entire file.

Sorted File

Records are maintained in sorted order, for example by:

Transaction_ID

This provides efficient searching and range queries.

However, frequent daily insertions can be expensive because records may need to be moved or the file reorganized.

Hashed File

A hash function can distribute transactions using a key such as:

Transaction_ID

Example:

$h(\text{Transaction_ID}) = \text{Transaction_ID} \bmod M$

This provides very fast average exact-match searches.

However, collisions can occur, requiring overflow handling or chaining.

Best Choice

For a supermarket billing system with **1 million transactions every day**, I recommend:

Hashed File Organization + appropriate indexing

for frequent exact transaction lookups.

For example:

Transaction_ID



Hash Function



Bucket



Transaction Record

If the system also requires frequent reports such as:

Transactions between 10:00 AM and 12:00 PM

Sales between ₹10,000 and ₹20,000

then a **B+ Tree index on transaction date/time** should additionally be used.

Final Comparison

Performance		
+-----+-----+		
Heap	Sorted	Hashed
Fast Insert	Fast Search	Fast Exact Search
Slow Search	Range Query	Collision Handling

Conclusion

For the given supermarket billing workload, **Hashed File Organization is the best primary choice for fast transaction-ID retrieval**, because it provides approximately **O(1) average search and insertion**. A **B+ Tree secondary index on transaction date/time** can be added when range-based sales reports are required.